

Assignment 1 Problem Statement

1 Overview

In this assignment, you will solve a nurse scheduling problem using constraint satisfaction methods. Consider a hospital where the administration needs to create a roster that satisfies the hospital's staffing requirements while also respecting the constraints and availability of the nurses. Your task is to automate this scheduling process.

The assignment is divided into two parts. Part A considers a task with hard constraints on factors such as workload, nurse availability, and the types of nurses required. Part B extends the task by introducing soft constraints that control how the shifts of a nurse are distributed throughout the roster.

2 Problem Setting

Throughout the assignment, assume the following:

- The roster is to be created for $\mathbf{D}$ days. Each day can be a surgical day (denoted by S) or a general day (denoted by G).
- The hospital has a total of $\mathbf{N}$ nurses. There are two types of nurses: surgical nurses and general nurses. The number of surgical nurses is denoted by $\mathbf{N}_s$, and the number of general nurses is denoted by $\mathbf{N}_g = \mathbf{N} - \mathbf{N}_s$.
- Nurses are of two types (based on their training). Surgical nurses can attend to surgeries *and* general patient dealing. The second group is General nurses who only focus on general patient dealing.
- Each nurse can be assigned one of three types of shifts each day: morning (denoted by M), afternoon (denoted by A), or evening (denoted by E).
- A surgical shift starts in the morning and extends till afternoon. Hence, a surgical nurse when assigned to a surgical shift (denoted by B) must be on duty for the morning and afternoon slots (as they need to perform care duty after the surgery also).
- If a nurse is not assigned to any shift, they are assigned rest (denoted by R).
- The hospital requires an exact total of $\mathbf{m}$ nurses in the morning shift, $\mathbf{a}$ nurses in the afternoon shift, and $\mathbf{e}$ nurses in the evening shift on each day, irrespective of the type of the day.

3 Part A

The nurse roster must satisfy the following hard constraints. A roster is valid if and only if all of these constraints are satisfied.

- **H1:** Every nurse is assigned exactly one shift or rest on every day, with the assignment drawn from $\{M, A, E, R, B\}$. Shift B can only be assigned to a surgical nurse. General nurses cannot be assigned to the shift B
- **H2:** No nurse works two consecutive morning shifts.
- **H3:** No nurse works a morning shift immediately after an evening shift.
- **H4:** The requirement of $\mathbf{m}$ nurses in the morning shift, $\mathbf{a}$ nurses in the afternoon shift, and $\mathbf{e}$ nurses in the evening shift is satisfied exactly on each day.
- **H5:** No nurse works more than five consecutive days. Equivalently, for every nurse, every window of six consecutive days must contain at least one rest day. If the number of days is less than equal to five then the rest day is not a necessity.
- **H6:** A surgical nurse assigned to shift B on a day cannot be assigned to the morning (M) or afternoon (A) shift on the following day.
- **H7:** Every surgical day has at least one surgical nurse assigned to shift B .
- **H8:** Every nurse can work a maximum of $\mathbf{K}$ shifts across all $\mathbf{D}$ days.
- **H9:** Every nurse has preapproved leave days. A nurse must be assigned rest (R) on every day for which they are on leave.

4 Part B

In Part B, we introduce an additional soft constraint that encourages the shifts of each nurse to be evenly distributed among morning, afternoon, and evening shifts. Given a valid roster from Part A, we prefer rosters in which each nurse's shifts are as evenly distributed as possible, i.e., asking a nurse to do many morning shifts and very few evening shifts will not be considered good.

Formally, let us capture this notion by defining a cost as follows. For nurse i , let $C_{i,M}$, $C_{i,A}$, and $C_{i,E}$ denote the number of morning, afternoon, and evening shifts worked by the nurse over the duration of $\mathbf{D}$ days. Note that a surgery shift (B) contributes one to both $C_{i,M}$ and $C_{i,A}$ and the rest (R) days are not considered here.

Let

$$\mu_i = \frac{1}{3} (C_{i,M} + C_{i,A} + C_{i,E}).$$

Let the cost of a full roster is defined as

$$\mathcal{C} = 9 \sum_{i=0}^{N-1} \frac{1}{3} [(C_{i,M} - \mu_i)^2 + (C_{i,A} - \mu_i)^2 + (C_{i,E} - \mu_i)^2].$$

Here, one is adding the deviation from the mean for all nurses and the deviation for a nurse is averaged over the three shifts. Finally, there is a scaling done. Note that a *lower* value of $\mathcal{C}$ corresponds to a better solution. In particular, when a nurse has an equal number of morning, afternoon, and evening shifts, their contribution to $\mathcal{C}$ is 0. Note that $\mathcal{C}$ is always an integer.

5 Submission Guidelines

As this is a large class, the assignments will be automatically evaluated. Therefore, please follow the submission guidelines carefully. Any deviation from the specified format may halt the evaluation process and may result in a penalty.

5.1 Starter Code and Environment Setup

You are provided with the starter code on Moodle. The structure of the starter code is as follows:

```
A1-starter-code/  
├── verify.py  
├── visualizer.py  
├── part_a.py  
├── part_b.py  
├── sample_test_cases  
│   ├── test1.csv  
│   ├── test2.csv  
│   └── test3.csv
```

You should only modify the files highlighted in red. Your solution for Part A should be implemented in `part_a.py`, while your solution for Part B should be implemented in `part_b.py`. The purpose of the remaining files will be explained in the subsequent sections.

Your code will be evaluated in a protected Conda environment. Therefore, make sure that your code runs correctly in the following environment:

```
conda create -n ai_a1 python=3.10  
conda activate ai_a1
```

Please do not use any additional python packages. The evaluation will be performed on a single CPU machine with 64 GB of RAM. **Do not use multithreading, multiprocessing, or subprocesses in your code.**

5.2 Input Output Format

Your program will be run with two command-line arguments:

```
python part_a.py <input_csv_path> <output_json_path>
```

For example:

```
python part_a.py test.csv solution.json
```

The first argument is the path to the input CSV file, and the second argument is the path to the output JSON file. The same format is used for Part B. The input CSV contains a header row followed by a single value row with the following 11 columns:

`N,D,N_s,N_g,m,a,e,T,days,K,leaves`

Symbol	Definition
N, D	The number of nurses and the number of days
N_s, N_g	The number of surgical and general nurses; $N_s + N_g = N$
m, a, e	The exact cover (number of nurses) required on any day
T	The time budget for providing a solution to a test problem (in seconds)
days	A string of length D over $\{G, S\}$ indicating which days are general and which days are surgical
K	The maximum shifts allowed for any nurse overall
leaves	A string of length ND encoding nurse availability. The i -th block of D characters corresponds to nurse i . L indicates that the nurse is on leave, while W indicates that the nurse is available.

Note that nurses and days are 0-indexed. Nurses with indices 0 through $N_s - 1$ are surgical nurses, while nurses with indices N_s through $N - 1$ are general nurses. Days are indexed from 0 through $D - 1$. The starter code provided to you handles reading the CSV input.

Your program must write exactly one JSON object to the specified output JSON path. For a valid roster, the object must contain all $(N \times D)$ nurse to shift assignment for all days using keys of the form "Ni-j" and values from "M", "A", "E", "R", or "B". If no valid roster exists, write the empty JSON object {}.

5.3 Verifier and Visualizer

The provided `verifier.py` script checks whether the roster produced by your program is valid and satisfies all the hard constraints. To verify a roster, run:

```
python verifier.py <input_csv_path> <output_json_path>
```

The provided `visualizer.py` script can be used to visualize a roster. To visualize a roster, run:

```
python visualizer.py <input_csv_path> <output_json_path> NUM_DAYS_PER_LINE
```

In both commands, `<input_csv_path>` is the path to the input CSV file, and `<output_json_path>` is the path to the corresponding output JSON file. You can adjust `NUM_DAYS_PER_LINE` if the output of the visualizer does not fit into your screen.

5.4 Worked Out Example

Example 1: Consider a instance with $N = 4$, $D = 3$, $m = a = e = 1$, $N_s = 1$, $K = 3$, `days = 'SGG'`, and `leaves = WWWWLW` then one of the valid solution is:

```
{"N0_0": "B", "N0_1": "R", "N0_2": "M",
 "N1_0": "E", "N1_1": "A", "N1_2": "R",
 "N2_0": "R", "N2_1": "M", "N2_2": "A",
 "N3_0": "R", "N3_1": "E", "N3_2": "E"}
```

First, note that the printing is with days is shown horizontally and the nurse allocations are displayed vertically. The indexing is from zero.

Examine the resulting roster for the first day (indexed as 0). Here, the nurse 0 is assigned to the B shift on day 0 which is supposed to be a surgery day. The surgery shift (B) consists of consecutive morning and afternoon shifts and therefore counts towards both an M and an A shift for the staffing requirements of day 0. Thus, the $m = 1$ and $a = 1$ requirements for day 0 are both satisfied by nurse 0. The $e = 1$ requirement is satisfied by the second nurse (indexed as 1), who is assigned the E shift. Since the nurse requirements for morning, afternoon and evening shifts is now met, the rest of the nurses are assigned for the rest slot.

Now, consider the allocations for the third day (indexed as 2). Here, the second nurse (indexed as 1) is supposed to be on a pre-approved leave on the third day (indexed as 2). This was specified in the input string `WWWLWWWWW` where the relevant string for the second nurse is `WL` indicating that the first two days are working and the third day is the leave day. Now, in the roster, this nurse has been correctly allocated rest (R) on the third day (indexed as 2).

Example 2: Consider an instance with $N = 4$, $D = 2$, $m = a = e = 1$, $N_s = 1$, $K = 2$, `days = 'SG'`, and `leaves = 'WWWWWWW'`. One valid solution is:

```
{"N0_0": "B", "N0_1": "R",
 "N1_0": "E", "N1_1": "A",
 "N2_0": "R", "N2_1": "M",
 "N3_0": "R", "N3_1": "E"}
```

Observe that on the first day, the first nurse (indexed as 0) is assigned the surgery shift (B). This consists of both morning and the afternoon shift contributing two slots to the maximum of ($K = 2$) shifts allowed per nurse. Hence, the first nurse (indexed as 0) has already reached the maximum number of allowed shifts for that day and hence cannot do the evening shift (E) and hence has been assigned for rest (R).

Example 3: Consider an instance with $N = 4$, $D = 6$, $m = a = e = 1$, $N_s = 1$, $K = 5$, `days = 'SGGGGG'`, and no leaves for any nurse. One valid solution is:

```
{"N0_0": "B", "N0_1": "R", "N0_2": "R", "N0_3": "R", "N0_4": "R", "N0_5": "E",
 "N1_0": "E", "N1_1": "E", "N1_2": "E", "N1_3": "E", "N1_4": "E", "N1_5": "R",
 "N2_0": "R", "N2_1": "M", "N2_2": "A", "N2_3": "M", "N2_4": "A", "N2_5": "M",
 "N3_0": "R", "N3_1": "A", "N3_2": "M", "N3_3": "A", "N3_4": "M", "N3_5": "A"}
```

Observe that the second nurse (indexed as 1) is assigned rest (R) on the fifth day (indexed as 4). Otherwise, the nurse would have worked for six consecutive days (violating the constraint that no nurse may work more than five consecutive days). Note that the six-day window from day 0 to day 5 must contain at least one rest day.

5.5 Submission Format

This assignment is to be done individually or in pairs. The choice is entirely yours. If the assignment is being done in pairs then the partner is to be selected only within your class.

- Please submit only your best algorithm for each part.

- Please succinctly describe the core ideas of your algorithms in the text file called `report.txt` (upto 2 pages in standard 11 point Arial font). The first line of the report should list the name and entry number of student(s) who submit the assignment.
- The assignment is to be submitted on Gradescope. **Exactly ONE of the team members** needs to make the submission. All registered students have already been added to Gradescope. The details are given below.
 - You need to upload a single ZIP file named `submission.zip`. Upon unzipping, this zip should create a folder containing `part_a.py`, `part_b.py`, `report.txt` and `group.txt`. Do not rename the zip file.
 - The `group.txt` should include the entry numbers of all the group members (one per line). If you are working alone, simply include your own entry numbers. A sample ‘`group.txt`’ is given below.


```
2020CSZ1234
2020CSZ5678
```
- Gradescope will run a sanity check for your submission. Sanity check will test your code for correct input/output formats and illegal imports and will run your code with dummy inputs. Your submission must pass this sanity test. Submissions failing the sanity check will not be evaluated.
- If you are working in pairs, you **must** select the partner using the **Group Members** option after uploading the submission in Gradescope.

5.6 Evaluation

Part A will be evaluated based on the efficiency of generating valid rosters. For Part B, evaluation will consider both the efficiency of generating valid rosters and the cost of the resulting roster under the soft constraint. During evaluation, a maximum time limit of 20 minutes will be imposed on instances with $N \leq 50$ and $D \leq 30$. Note that the part A will carry roughly 80% weightage and the part B will carry roughly 20% weightage.

5.7 Implementation Hints

Convert the nurse scheduling problem into a Constraint Satisfaction Problem (CSP). You may start with a basic backtracking search algorithm and improve it using heuristics such as Minimum Remaining Values (MRV), Least Constraining Value (LCV), forward checking, or arc consistency.

For Part B, the objective is to find a solution that satisfies all the hard constraints while maximizing an overall score determined by the soft constraints. Unlike hard constraints, violations of soft constraints do not make a solution invalid; instead, they affect its score. Explore how soft constraints can be incorporated into the search process to efficiently find a high-scoring solution.